



Libreria Grafica WebGL



<https://www.khronos.org/webgl/>



API WebGL

L'API WebGL è integrata in HTML 5, quindi non sono necessarie installazioni aggiuntive.

Le applicazioni WebGL sono scritte in JavaScript e quindi non è necessario compilare.

Per scrivere ed eseguire un'applicazione WebGL, tutto ciò che serve è un **editor di testo** e un **web browser**.

WebGL è open source (Kronos Group).

Supporto Mobile: WebGL supporta anche i browser su Mobile come iOS Safari e Chrome per Android.



HTML5+canvas+contesto webgl

Abbiamo già visto l'elemento `canvas` e il contesto 2d; ora vediamo il contesto WebGL.

Per creare un contesto di rendering WebGL sull'elemento canvas, è necessario passare la stringa `experimental-webgl` (od anche solo `webgl`), anziché `2d`, al metodo `canvas.getContext()`.

```
<!DOCTYPE html>
<html>
  <canvas id='my_canvas'></canvas>
  <script>
    var canvas = document.getElementById('my_canvas');
    var gl = canvas.getContext('webgl');
    gl.clearColor(0.9,0.9,0.8,1);
    gl.clear(gl.COLOR_BUFFER_BIT);
  </script>
</html>
```



Sistema di Coordinate WebGL

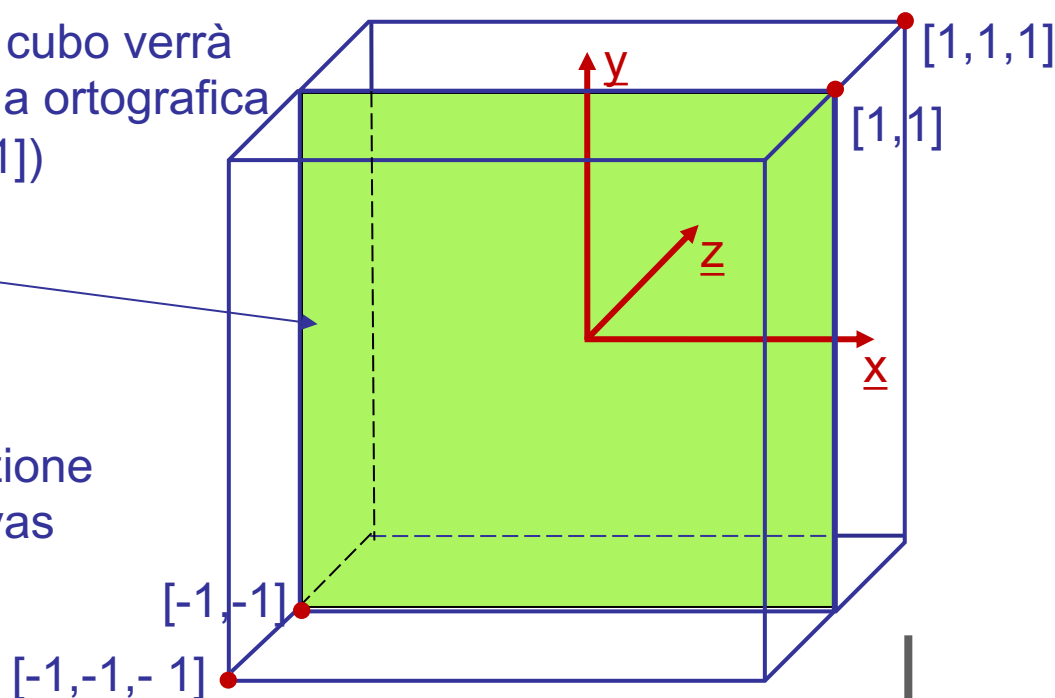
In WebGL il sistema di coordinate è 3D, l'asse z indica la profondità, l'osservatore guarda nel verso positivo dell'asse z). Le coordinate sono limitate fra $[-1,1] \times [-1,1] \times [-1,1]$.

WebGL non visualizza nulla se fuori da questo cubo.

Una geometria definita in questo cubo verrà visualizzata in proiezione parallela ortografica sul piano xy (**window** $[-1,1] \times [-1,1]$)

piano di proiezione
e window

Poi viene applicata la trasformazione
window $\rightarrow$ viewport/canvas





API WebGL

Dopo aver ottenuto il contesto WebGL dell'elemento canvas, è possibile iniziare a disegnare elementi grafici utilizzando l'API WebGL.

Una geometria 3D si definisce tramite una lista di vertici ed una lista di facce triangolari; per es. la seguente è una faccia triangolare 3D

```
var vertices = [ -1,-1,-1,  1,-1,-1,  1, 1,-1 ];
```

Buffer Object

I buffer sono le aree di memoria WebGL, su GPU, in cui devono essere memorizzati i vertici e i loro attributi. Esistono vari buffer, i più importanti sono il **Vertex Buffer Object (VBO)** e l'**Index Buffer Object (IBO)** e vengono utilizzati per contenere rispettivamente la geometria (i vertici e i loro attributi) e la loro indicizzazione.

Dopo aver definito/memorizzato i vertici in array JavaScript, li passiamo alla GPU memorizzandoli in questi Buffer Object di WebGL.



API WebGL

Per 'disegnare' oggetti 2D o 3D, l'API WebGL fornisce due metodi:

`drawArrays( )` e `drawElements( )`

Questi due metodi accettano un parametro chiamato **mode** mediante il quale è possibile selezionare l'oggetto che si desidera 'disegnare'; le possibilità sono limitate a **punti**, **linee** e **triangoli**.

```
drawArrays(gl.POINTS, ... , ...);  
drawArrays(gl.LINES, ... , ...);  
drawArrays(gl.LINE_STRIP, ... , ...);  
...  
drawArrays(gl.TRIANGLES, ... , ...);  
drawArrays(gl.TRIANGLE_STRIP, ... , ...);  
....
```

Invocando da JavaScript uno di questi metodi, inizia l'esecuzione della pipeline grafica o di rendering su GPU che produrrà un'immagine sullo schermo (all'interno del canvas).



Shader Programs

WebGL utilizza ES SL (Embedded System Shader Language) che gira su GPU; scriveremo i programmi per disegnare quanto memorizzato nei Buffer usando **shader program**.

Gli shader sono i programmi per GPU; il linguaggio utilizzato è GLSL (OpenGL Shader Language). In questi shader program, definiamo esattamente come i vertici, i colori, le trasformazioni, ecc., interagiscono tra loro per creare un'immagine.

In breve, è un codice che implementa algoritmi per ottenere i pixel dell'immagine 2D a partire da una geometria 2D o 3D.

Esistono fondamentalmente due tipi di shader program:

Vertex Shader e **Fragment Shader**



Vertex Shader

Vertex shader è il codice del programma chiamato su ogni vertice.

Viene utilizzato per trasformare (spostare) da una posizione ad un'altra la geometria, vertice per vertice.

Gestisce i vertici e i loro attributi (dati per vertice) come le coordinate dei vertici, i colori, le normali, le coordinate texture.

Nel codice GLSL del vertex shader, i programmatori devono definire gli **attribute** per gestire i dati per vertice.

Ogni **attribute** si riferisce ad un **Vertex Buffer Object**.



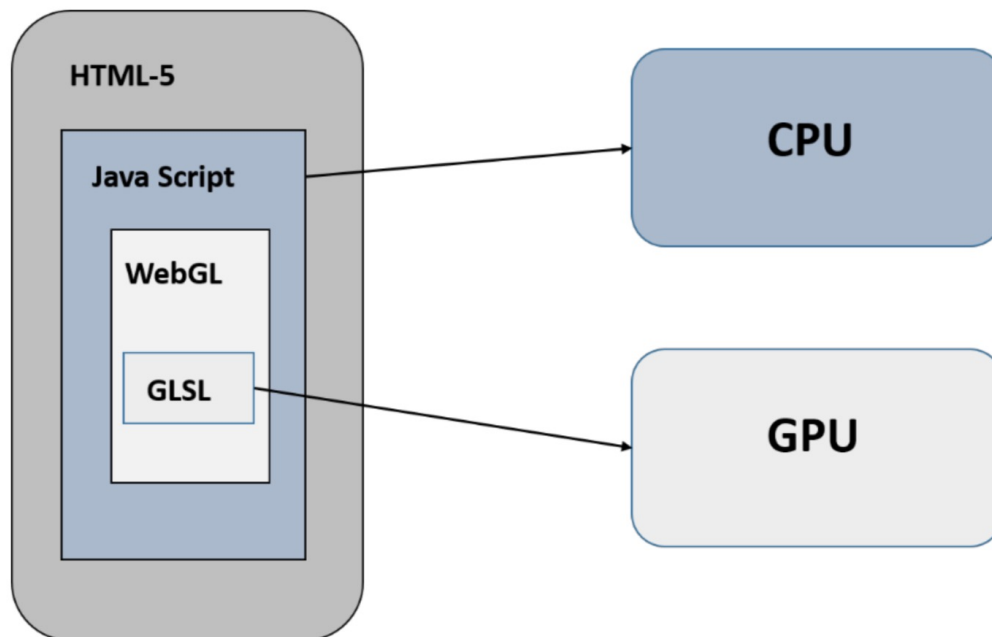
Fragment Shader

Una geometria viene trasformata in triangoli di pixel e ciascuno di questi è chiamato **fragment**.

Fragment shader è il codice che viene eseguito su ogni pixel di ogni triangolo.

Viene scritto per calcolare e definire il colore dei singoli pixel.

Vediamo un Esempio di codice WebGL



- JavaScript serve per comunicare con la CPU
- GLSL serve per comunicare con la GPU

Codice **Triangle.html**
nella cartella **HTML5_webgl_1**



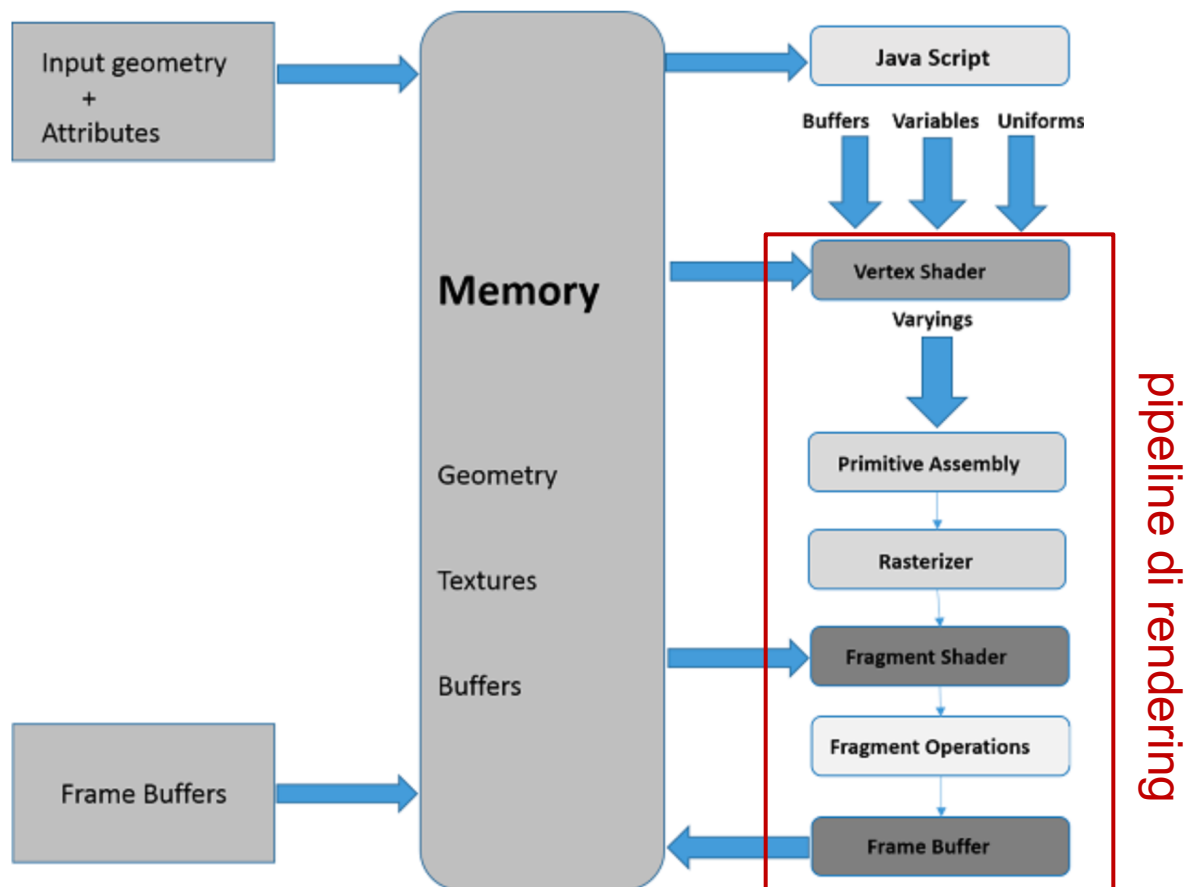
Applicazione WebGL

Vediamo un'interessante applicazione che simula l'esecuzione di un codice WebGL e spiega cosa succede all'interno della GPU a seguito di ogni istruzione del codice.

Pagina Web del corso, Siti, WebGL Fundamentals,
WebGL State Diagram:
codice demo **Triangle**

<https://webglfundamentals.org/webgl/lessons/resources/webgl-state-diagram.html?exampleId=triangle#no-help>

Pipeline Grafica di WebGL



Nelle slide successive verrà spiegato il ruolo di ogni passaggio nella pipeline di rendering in figura (i titoli delle slide ricalcano e seguono la pipeline di rendering mostrata a destra in figura)



JavaScript

Durante lo sviluppo di applicazioni WebGL, si scrive il codice nello Shading Language per comunicare con la GPU. Questo codice viene scritto all'interno di un codice JavaScript che include le seguenti azioni:

- **Inizializza WebGL**: JavaScript viene utilizzato per inizializzare il contesto WebGL.
- **Crea array**: si crea un array JavaScript per contenere i dati per la geometria.
- **Buffer Objects**: si creano buffer object (vertici, indici, ecc.) passando array come parametri.
- **Shaders**: si scrivono, compilano e linkano i programmi vertex shader e fragment shader.

continua



JavaScript

- **Attribute**: si possono creare attribute, abilitarli e associarli a Buffer Object.
- **Uniform**: si possono anche associare uniform.
- **Matrice di trasformazione**: si possono creare matrici di trasformazione.

Inizialmente si definiscono i dati per la geometria e si passano agli shader sotto forma di buffer.



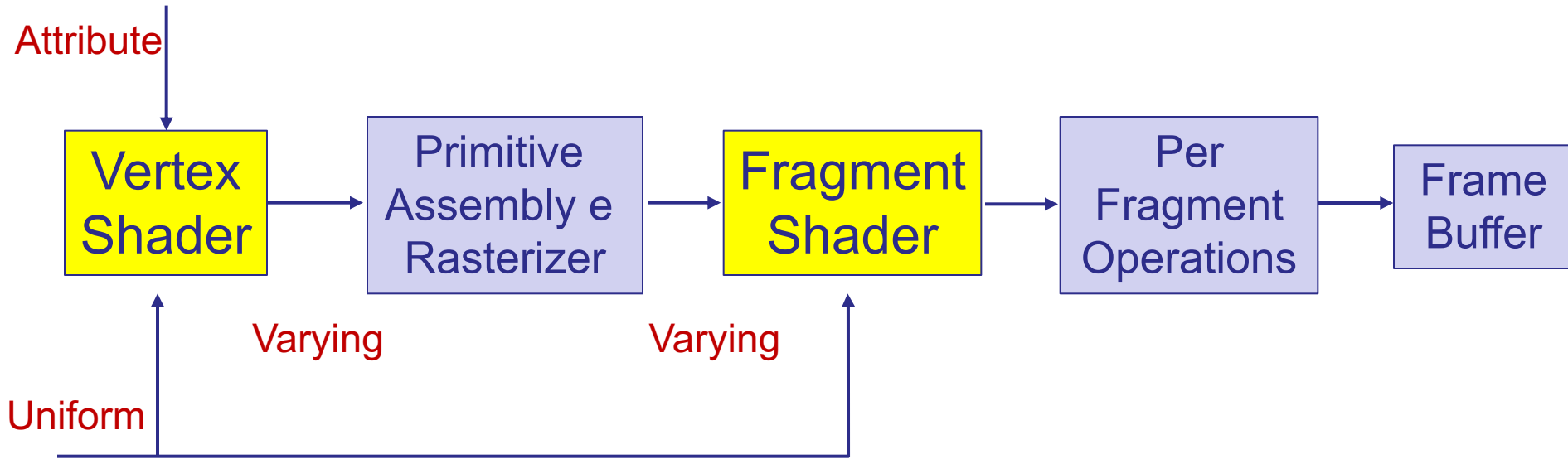
Variabili OpenGL ES SL

Per gestire i dati negli **shader program**, OpenGL ES SL fornisce tre tipi di variabili.

- Attribute**: sono le variabili che contengono i valori di input del programma Vertex Shader. Indicano il Vertex Buffer Object che contiene i dati dei vertici e i Buffer che contengono altre informazioni sui vertici. Ogni volta che viene richiamato il vertex shader, i valori attribute variano.
- Uniform**: sono le variabili che contengono i dati di input comuni per Vertex Shader e Fragment Shader, come le matrici di trasformazione, ecc.
- Varying**: sono le variabili utilizzate per passare i dati dal Vertex Shader al Fragment Shader.



PipeLine





Vertex Shader

Invocando i metodi `drawElements( )` e/o `drawArrays( )` inizia il processo di rendering:

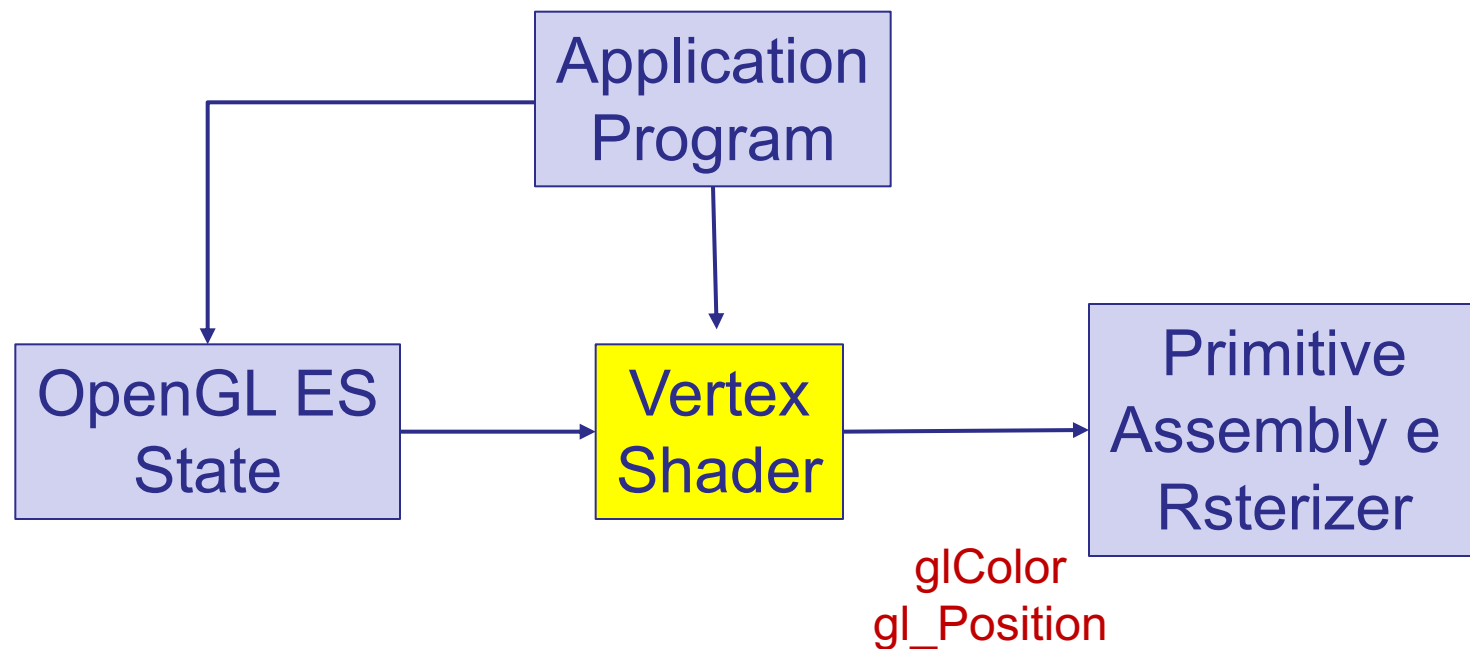
parte il **vertex shader** che viene eseguito per ciascun vertice presente nel Vertex Buffer Object;

calcola la posizione di ciascun vertice di un poligono primitivo (punti, linee e triangoli) e lo memorizza nella variabile `gl_Position` di WebGL;

calcola anche gli altri attributi come colore, coordinate texture, ecc. che sono usualmente associati a un vertice.



Architettura del Vertex Shader



La minima richiesta del Vertex Shader è di settare una posizione per il **vertex** assegnando un valore a `gl_Position`



Primitive Assembly e Rasterizer

Dopo aver calcolato la posizione e altri dettagli di ciascun vertice, la fase successiva è fissa e consiste nell'**assemblaggio dei vertici in triangoli**.

Ci sono due passaggi:

- **Culling**: inizialmente viene determinato l'orientamento (frontale o posteriore) del triangolo. Tutti quei triangoli con orientamento improprio che si trovano all'esterno del **frustum (*)** vengono eliminati.
- **Clipping**: se un triangolo è parzialmente al di fuori del **frustum**, la parte all'esterno viene rimossa.

Quindi i vertici dei nuovi triangoli vengono trasformati in coordinate schermo e passati al **rasterizzatore**. Nella fase di **rasterizzazione**, vengono determinati, per interpolazione, i pixel dell'immagine finale della primitiva.

(*)**frustum** (tronco di piramide) è il volume di spazio 3D che la "telecamera virtuale" riesce a vedere.



Primitive Assembly e Rasterizer

Per gli algoritmi di clipping applicati in 2D e in 3D vedere le slide:

[cg7.2_clipping_lines.ppt](#)

[cg7.3_clipping_polygons.ppt](#)

Per l'algoritmo di rasterizzazione vedere le slide:

[cg7.4_rasterizing.ppt](#)



Fragment Shader

Il **fragment shader** prende:

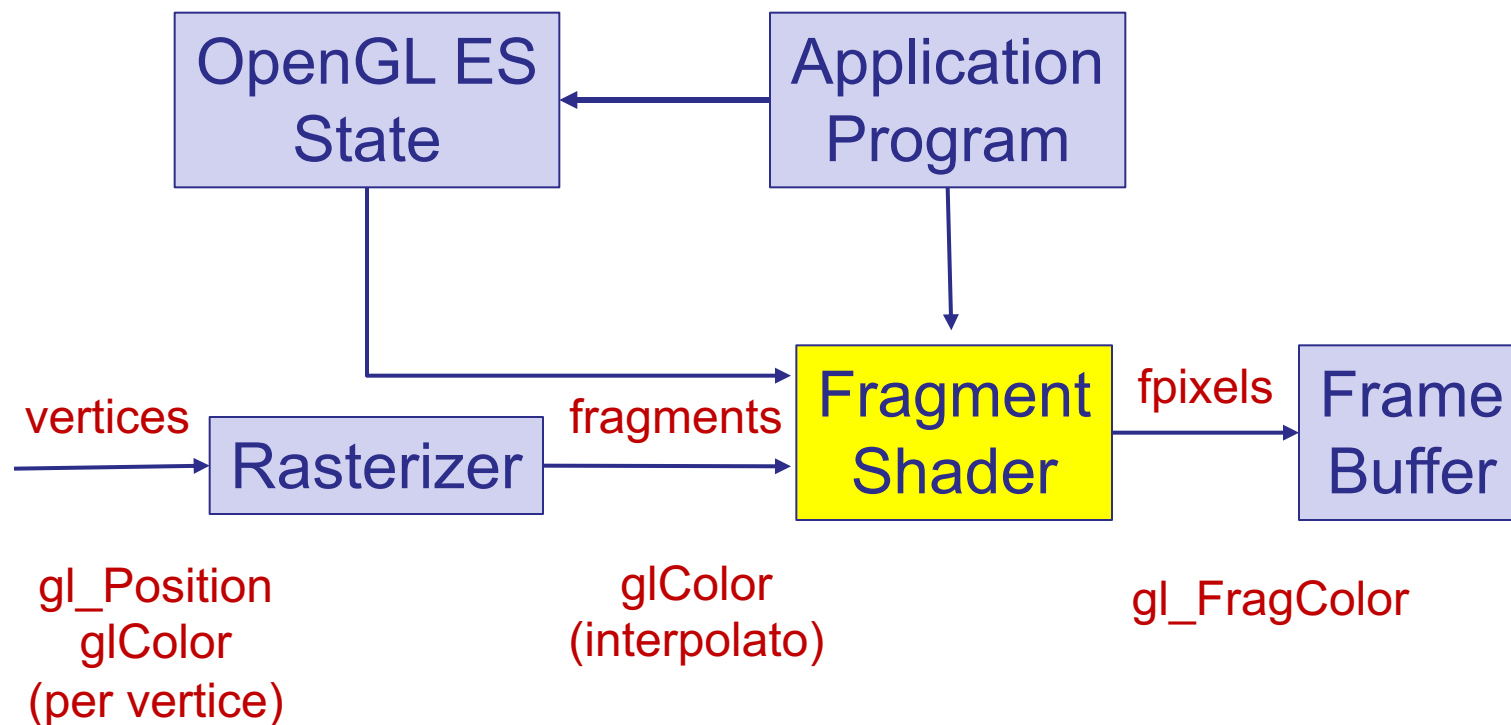
- dati dal vertex shader in variabili **Varying**,
- primitive dalla fase di rasterizzazione
- calcola i valori di colore per ciascun pixel tra i vertici

Il **fragment shader** memorizza i valori di colore di ogni pixel di ciascun triangolo.

È possibile accedere a questi valori di colore durante le operazioni sui fragment.



Architettura del Fragment Shader



La minima richiesta del Fragment Shader è di assegnare un colore al **fragment** settando **gl_FragColor** o scartandolo.

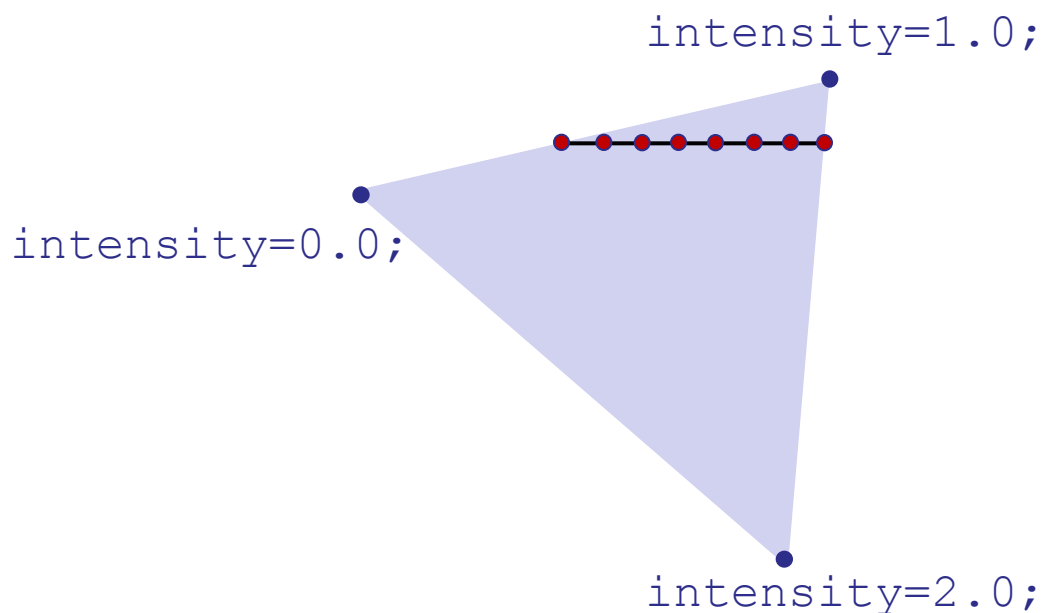


Qualificatore Varying

Il vertex shader scrive i valori nelle variabili e il fragment shader legge i valori interpolati linearmente fra i vertici.

varying float intensity;

Deve essere definito
nello stesso modo in
entrambi gli shader;



L'interpolazione è sempre attiva.



Fragment Operations

Dopo aver determinato il colore di ciascun pixel, vengono eseguite ulteriori e opzionali **operazioni sui pixel/fragment** (per **fragment operations**):

- Depth Test
- Blending
- Dithering
- Ecc.

Una volta elaborati tutti i pixel/fragment, viene formata e visualizzata sullo schermo un'immagine 2D.

Il **frame buffer** è la destinazione finale della pipeline di rendering.



VSCode ed Esempi

Per un esempio di codice spiegato ancor più nel dettaglio si vedano le slide:

`cg7.1_webgl_esempio.ppt`

Alla fine siete invitati ad aprire i codici della

Cartella: `HTML5_webgl_1`

nell'ordine indicato nel file `Readme`, quindi a provarli e a modificarli



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Giulio Casciola
Dip. di Matematica
giulio.casciola@unibo.it